

LAB PROGRAMS

The primary difference between standard C `clock()` and OpenMP's `omp_get_wtime()` lies in **what type of time they measure**:

The primary difference between standard C <code>clock()</code> and OpenMP's <code>omp_get_wtime()</code> lies in what type of time they measure:		
Feature	<code>clock()</code> (<time.h>)	<code>omp_get_wtime()</code> (<omp.h>)
What it measures	CPU Time (Processor Time) across all cores combined	Wall-Clock Time (Real-World Elapsed Time)
Multithreading Behavior	Sums up CPU time spent across every active thread	Tracks the real elapsed time from start to finish on a clock
Suitability for OpenMP	Incorrect (leads to artificial skewing)	Correct standard for parallel speedup measurement
Return Unit	Ticks (requires division by <code>CLOCKS_PER_SEC</code>)	Double-precision floating-point seconds directly

What Causes `clock()` to "Skew" Parallel Measurements?

When evaluating parallel computing performance, the objective is to measure **Wall-Clock Time**: how many real-world seconds it takes from when the task begins to when the task finishes.

`clock()` does not measure wall-clock time; it returns the total number of CPU clock cycles used by the process across all CPU cores.

The Problem: Summing Thread Times

When code executes on multiple threads concurrently, `clock()` adds up the execution time of **every single core**:

$$\text{Total CPU Time} \approx \text{Time per core} \times \text{Number of active cores}$$

Concrete Example:

Suppose a task takes **4 seconds** sequentially on 1 core.

- Sequential Execution (1 Core):**
 - Wall-Clock Time:** 4 seconds.
 - `clock()` **reading:** 4 CPU seconds.
- Parallel Execution (4 Cores running simultaneously):**

- Each core finishes its quarter of the workload in **1 second of real time**.
- **Wall-Clock Time** (`omp_get_wtime()`): **1.0 second** (a 4x speedup).
- **CPU Time** (`clock()`): Adds up Core 0 (1s) + Core 1 (1s) + Core 2 (1s) + Core 3 (1s) + thread overhead $\approx$ **4.1 seconds**.

Why This Skews Results

If you measure the parallel section with `clock()`, the output will display **~4.1 seconds** instead of **1.0 second**.

This makes parallel code appear **equal to or slower than serial code**, completely hiding the true performance gain achieved through multithreading.

Why `omp_get_wtime()` is the Correct Choice

- It functions like a real-world stopwatch.
- It measures the time elapsed from Point A to Point B on the wall clock, irrespective of how many CPU cores or threads were active during that interval.
- It provides high-resolution, double-precision timestamps in seconds, making speedup calculation straightforward:

$$\text{Speedup} = \frac{\text{Serial Elapsed Time (via } \text{omp_get_wtime}())}{\text{Parallel Elapsed Time (via } \text{omp_get_wtime}())}$$

Experiment 1: Parallel Merge Sort using OpenMP

Problem Statement

Write an OpenMP program to sort an array of n elements using both sequential and parallel Merge Sort (using section). Record the difference in execution time.

1. Executive Summary & Core Concept

Merge Sort is a classic **Divide and Conquer** algorithm:

1. **Divide:** Recursively split an array of n elements into two halves at index $\text{mid} = \text{left} + \frac{\text{right} - \text{left}}{2}$.
2. **Conquer:** Sort both halves independently until sub-arrays contain a single element.
3. **Combine (merge):** Stitch two sorted sub-arrays together in sorted ascending order.

How OpenMP Parallelizes Merge Sort

- In a standard sequential merge sort, the CPU sorts the left sub-array completely before starting the right sub-array.

- In OpenMP, #pragma omp parallel sections assigns sorting the left half to **Thread 1** and sorting the right half to **Thread 2** concurrently on separate CPU cores.

2. Complete C Source Code

C

```
#include <stdio.h>
```

```
#include <stdlib.h>
```

```
#include <omp.h>
```

```
#define SIZE 100000
```

```
// Combines two sorted sub-arrays into one sorted sequence
```

```
void merge(int arr[], int left, int mid, int right)
```

```
{
```

```
    int i, j, k;
```

```
    int n1 = mid - left + 1;
```

```
    int n2 = right - mid;
```

```
    int *L = (int *)malloc(n1 * sizeof(int));
```

```
    int *R = (int *)malloc(n2 * sizeof(int));
```

```
    for (i = 0; i < n1; i++)
```

```
        L[i] = arr[left + i];
```

```
    for (j = 0; j < n2; j++)
```

```
        R[j] = arr[mid + 1 + j];
```

```
    i = 0;
```

```
    j = 0;
```

```
    k = left;
```

```
    while (i < n1 && j < n2) {
```

```
        if (L[i] <= R[j]) {
```

```
            arr[k++] = L[i++];
```

```
        } else {
```

```
            arr[k++] = R[j++];
```

```
        }
```

```

    }

    while (i < n1)
        arr[k++] = L[i++];

    while (j < n2)
        arr[k++] = R[j++];

    free(L);
    free(R);
}

// Sequential recursive merge sort
void serialMergeSort(int arr[], int left, int right)
{
    if (left < right)
    {
        int mid = left + (right - left) / 2;
        serialMergeSort(arr, left, mid);
        serialMergeSort(arr, mid + 1, right);
        merge(arr, left, mid, right);
    }
}

// Parallel recursive merge sort using OpenMP sections
void parallelMergeSort(int arr[], int left, int right, int depth)
{
    if (left < right)
    {
        int mid = left + (right - left) / 2;

        if (depth <= 4)
        {
            #pragma omp parallel sections
            {
                #pragma omp section
                parallelMergeSort(arr, left, mid, depth + 1);
            }
        }
    }
}

```

```

        #pragma omp section
        parallelMergeSort(arr, mid + 1, right, depth + 1);
    }
}
else
{
    serialMergeSort(arr, left, mid);
    serialMergeSort(arr, mid + 1, right);
}

merge(arr, left, mid, right);
}
}

```

```

int main()
{
    int *arr_serial = (int *)malloc(SIZE * sizeof(int));
    int *arr_parallel = (int *)malloc(SIZE * sizeof(int));

    for (int i = 0; i < SIZE; i++)
    {
        int val = rand() % 100000;
        arr_serial[i] = val;
        arr_parallel[i] = val;
    }

    // Enable nested multithreading
    omp_set_nested(1);

    // Measure Sequential Execution Time
    double start_serial = omp_get_wtime();
    serialMergeSort(arr_serial, 0, SIZE - 1);
    double end_serial = omp_get_wtime();
    double time_serial = end_serial - start_serial;

    // Measure Parallel Execution Time

```

```

double start_parallel = omp_get_wtime();
parallelMergeSort(arr_parallel, 0, SIZE - 1, 0);
double end_parallel = omp_get_wtime();
double time_parallel = end_parallel - start_parallel;

printf("\n-----\n");
printf("Array Size          : %d elements\n", SIZE);
printf("Serial Merge Sort Time : %.6f seconds\n", time_serial);
printf("Parallel Merge Sort Time : %.6f seconds\n", time_parallel);
printf("Speedup Achieved       : %.2fx\n", time_serial / time_parallel);
printf("-----\n");

free(arr_serial);
free(arr_parallel);

return 0;
}

```

3. Detailed Line-by-Line & Block-by-Block Logic Breakdown

Block A: Headers and Definitions

- `#include <stdio.h>`: Provides standard I/O functions (printf).
- `#include <stdlib.h>`: Provides dynamic memory management (malloc, free) and random number generation (rand).
- `#include <omp.h>`: Imports the OpenMP runtime API (omp_get_wtime(), omp_set_nested()).
- `#define SIZE 100000`: Sets the constant array size to 100,000 integers.

Block B: The merge() Function

1. Calculating Partition Sizes

C

```
int n1 = mid - left + 1;
```

```
int n2 = right - mid;
```

- **n1**: The count of elements in the left partition (left to mid).
 - *Example*: If left = 0 and mid = 3, indices are 0, 1, 2, 3. Elements count = $3 - 0 + 1 = 4$.
- **n2**: The count of elements in the right partition (mid + 1 to right).
 - *Example*: If mid = 3 and right = 7, indices are 4, 5, 6, 7. Elements count = $7 - 3 = 4$.

2. Allocating Temporary Buffers

C

```
int *L = (int *)malloc(n1 * sizeof(int));
```

```
int *R = (int *)malloc(n2 * sizeof(int));
```

- Allocates temporary scratchpad buffers in RAM to hold copies of the left and right subarrays while comparing elements.

3. Copying Elements to Scratchpads

C

```
for (i = 0; i < n1; i++)
```

```
    L[i] = arr[left + i];
```

```
for (j = 0; j < n2; j++)
```

```
    R[j] = arr[mid + 1 + j];
```

- L[i] receives values from the left section: arr[left + 0], arr[left + 1], etc.
- R[j] receives values starting right after mid: arr[mid + 1 + 0], arr[mid + 1 + 1], etc.

4. Merging Elements in Sorted Order

C

```
i = 0; j = 0; k = left;
```

```
while (i < n1 && j < n2) {
```

```
    if (L[i] <= R[j]) {
```

```
        arr[k++] = L[i++];
```

```
    } else {
```

```
        arr[k++] = R[j++];
```

```
    }
```

```
}
```

- i: Tracks position in L.
- j: Tracks position in R.
- k: Tracks write index in arr starting at left.
- Compares L[i] and R[j]. The smaller number is copied into arr[k], and the corresponding pointers advance.

5. Flushing Remaining Elements & Cleanup

C

```
while (i < n1) arr[k++] = L[i++];
```

```
while (j < n2) arr[k++] = R[j++];
```

free(L); free(R);

- Copies any leftover elements from either buffer into arr.
- Deallocates L and R memory to prevent memory leaks.

Block C: The Parallel Merge Sort (parallelMergeSort)

C

```
void parallelMergeSort(int arr[], int left, int right, int depth)
```

```
{
    if (left < right)
    {
        int mid = left + (right - left) / 2;

        if (depth <= 4)
        {
            #pragma omp parallel sections
            {
                #pragma omp section
                parallelMergeSort(arr, left, mid, depth + 1);

                #pragma omp section
                parallelMergeSort(arr, mid + 1, right, depth + 1);
            }
        }
        else
        {
            serialMergeSort(arr, left, mid);
            serialMergeSort(arr, mid + 1, right);
        }

        merge(arr, left, mid, right);
    }
}
```

- $\text{mid} = \text{left} + (\text{right} - \text{left}) / 2$: Computes the middle index safely without arithmetic overflow.
- $\text{depth} \leq 4$ (Depth Thresholding):

- Spawning threads incurs an operating system overhead.
 - For small sub-arrays, the overhead of creating threads exceeds the sorting computation time.
 - When depth ≤ 4 , up to $2^4 = 16$ parallel branch tasks execute concurrently. Beyond depth 4, the algorithm falls back to `serialMergeSort()`.
- `#pragma omp parallel sections`: Defines a region with independent tasks executed in parallel by available threads.
 - `#pragma omp section`: Marks an individual concurrent task (one for the left half, one for the right half).

Block D: The `main()` Driver Function

C

```
omp_set_nested(1);
```

- Enables nested parallelism so recursive parallel sections can create sub-threads.

C

```
double start_serial = omp_get_wtime();
```

```
serialMergeSort(arr_serial, 0, SIZE - 1);
```

```
double end_serial = omp_get_wtime();
```

- `omp_get_wtime()` measures elapsed **wall-clock time** in seconds.
- `clock()` from `<time.h>` measures total CPU time across all cores, which distorts multithreaded speedup metrics.

4. Ubuntu Terminal Commands

- **Compile:**

Bash

```
gcc -fopenmp program1.c -o program1
```

- **Run:**

Bash

```
./program1
```

5. Expected Output

Plaintext

```
-----
```

```
Array Size          : 100000 elements
```

```
Serial Merge Sort Time : 0.021840 seconds
```

Parallel Merge Sort Time : 0.007120 seconds

Speedup Achieved : 3.07x

EXPERIMENT 2: STATIC LOOP SCHEDULING USING OpenMP

1. Problem Statement & Executive Summary

Problem Statement

Write an OpenMP program that divides the iterations into chunks containing 2 iterations, respectively (`schedule(static, 2)`). Its input should be the number of iterations, and its output should be which iterations of the parallelized for loop are executed by which thread.

For example, if there are two threads and four iterations, the output might be the following:

a. Thread 0 : Iterations 0 – 1

b. Thread 1 : Iterations 2 – 3

Executive Summary & Core Concept

- In OpenMP, work-sharing loop constructs split the iterations of a for loop across multiple threads in a thread team.
- **Static Scheduling** (`schedule(static, 2)`): Divides the total iterations into contiguous chunks of size 2 and distributes them among the available threads in a round-robin cyclic order at compile/runtime setup.
- By calling `omp_get_thread_num()` inside the parallel loop alongside the loop counter `i`, we can directly observe which thread processes each iteration chunk.

2. Complete C Source Code

C

```
#include <stdio.h>
```

```
#include <omp.h>
```

```
int main()
```

```
{
```

```
    int n, thread;
```

```
    // User inputs total iterations and number of threads
```

```
    printf("Enter the number of tasks/iterations: ");
```

```

if (scanf("%d", &n) != 1 || n <= 0) {
    printf("Invalid number of iterations!\n");
    return 1;
}

```

```

printf("Enter the number of threads: ");
if (scanf("%d", &thread) != 1 || thread <= 0) {
    printf("Invalid number of threads!\n");
    return 1;
}

```

```

// Set the number of threads requested by the user
omp_set_num_threads(thread);

```

```

printf("\n-----\n");

```

```

// Parallel loop with static scheduling and chunk size of 2
#pragma omp parallel for schedule(static, 2)
for (int i = 0; i < n; i++)
{
    printf("Thread %d executes iteration %d\n", omp_get_thread_num(), i);
}

```

```

printf("-----\n");
return 0;
}

```

3. Detailed Line-by-Line & Block-by-Block Logic Breakdown

Block A: Headers and Variable Declarations

```

C
#include <stdio.h>
#include <omp.h>

int main()
{
    int n, thread;
    • #include <stdio.h>: Imports standard C input/output library functions like
      printf() and scanf().

```

- `#include <omp.h>`: Imports OpenMP functions such as `omp_set_num_threads()` and `omp_get_thread_num()`.
- `int n`: Variable used to store the total number of tasks/iterations that the user wants to run.
- `int thread`: Variable used to store the number of parallel threads requested by the user.

Block B: Reading Inputs & Configuring OpenMP Threads

C

```
printf("Enter the number of tasks/iterations: ");
scanf("%d", &n);
```

```
printf("Enter the number of threads: ");
scanf("%d", &thread);
```

```
omp_set_num_threads(thread);
```

- `scanf("%d", &n)`: Reads the integer value for total iterations from user terminal input.
- `scanf("%d", &thread)`: Reads the integer value for thread count from user terminal input.
- `omp_set_num_threads(thread)`: Dynamically sets the size of the OpenMP thread team to the exact number requested by the user before entering the parallel region.

Block C: The Parallel Loop with Static Scheduling

C

```
#pragma omp parallel for schedule(static, 2)
for (int i = 0; i < n; i++)
{
    printf("Thread %d executes iteration %d\n", omp_get_thread_num(), i);
}
```

- `#pragma omp parallel for`: Tells the compiler to create a team of threads and distribute the loop iterations among them.

- `schedule(static, 2)` (**How it distributes the iterations**):
 - Divides the loop iterations into chunks of **2 consecutive iterations**.
 - Chunks are assigned to threads in round-robin order:
 - **Chunk 1** (iterations 0, 1) → Assigned to **Thread 0**
 - **Chunk 2** (iterations 2, 3) → Assigned to **Thread 1**
 - If there are only 2 threads and $n = 8$:
 - **Chunk 3** (iterations 4, 5) → Wraps back to **Thread 0**
 - **Chunk 4** (iterations 6, 7) → Wraps back to **Thread 1**
- `omp_get_thread_num()`: Returns the unique integer ID (0,1, ...,thread – 1) of the specific thread executing that iteration.
- `i`: The current iteration index (0 to $n - 1$).
- `printf(...)`: Prints the executing thread's ID alongside its assigned iteration index.

4. Ubuntu Terminal Commands

- **Compile:**

Bash

```
gcc -fopenmp program2.c -o program2
```

- **Run:**

Bash

```
./program2
```

5. Expected Output

Case 1: 4 Iterations with 2 Threads (Matching Problem Example)

Plaintext

Enter the number of tasks/iterations: 4

Enter the number of threads: 2

```
-----
Thread 0 executes iteration 0
```

```
Thread 0 executes iteration 1
```

```
Thread 1 executes iteration 2
```

```
Thread 1 executes iteration 3
-----
```

Case 2: 8 Iterations with 2 Threads (Demonstrating Round-Robin Cycling)

Plaintext

Enter the number of tasks/iterations: 8

Enter the number of threads: 2

Thread 0 executes iteration 0

Thread 0 executes iteration 1

Thread 1 executes iteration 2

Thread 1 executes iteration 3

Thread 0 executes iteration 4

Thread 0 executes iteration 5

Thread 1 executes iteration 6

Thread 1 executes iteration 7

(Note: Because threads execute concurrently in parallel, the exact line-by-line print sequence in the terminal may interleave, but the iteration-to-thread allocation will strictly follow the 2-iteration chunks).

EXPERIMENT 3: FIBONACCI NUMBERS USING OpenMP TASKS

(Note: Numbered as Program 4 in your lab manual due to the inclusion of sample programs)

1. Problem Statement & Executive Summary

Problem Statement

Write an OpenMP program to calculate n Fibonacci numbers using tasks. Compare both serial and parallel calculation times.

Executive Summary & Core Concept

- The Fibonacci sequence is defined by the recurrence relation:

$$F(0) = 0, \quad F(1) = 1, \quad F(n) = F(n - 1) + F(n - 2)$$

- **OpenMP Tasks (#pragma omp task):** Unlike work-sharing loops (omp for), tasks are ideal for irregular, dynamic, or recursive computations. A task encapsulates a unit of work that can be executed independently by any available thread in the thread pool.
- **Synchronization (#pragma omp taskwait):** Ensures that the parent task waits until its explicitly spawned child tasks have finished computing before it adds the results together ($x + y$).

- `#pragma omp single`: Ensures that only one thread sets up the loop and spawns the initial tasks, preventing all threads from redundantly duplicating task generation.

Issues Identified & Corrected in the Manual Code:

1. **Missing Header Inclusions:** The manual code has incomplete `#include` directives. We need `<stdio.h>` for standard I/O and `<omp.h>` for OpenMP runtime routines.
2. **Task Synchronization Flow:** Cleaned up variable scoping and task waiting barriers to prevent race conditions when filling the array and accumulating recursive returns.

2. Complete C Source Code

C

```
#include <stdio.h>
```

```
#include <omp.h>
```

```
// Serial implementation of Fibonacci computation
```

```
void fib_serial(int n)
```

```
{
    int fib_numbers[n + 1];
    fib_numbers[0] = 0;
    fib_numbers[1] = 1;

    printf("Serial Fibonacci numbers : %d %d ", fib_numbers[0], fib_numbers[1]);
    for (int i = 2; i <= n; i++)
    {
        fib_numbers[i] = fib_numbers[i - 1] + fib_numbers[i - 2];
        printf("%d ", fib_numbers[i]);
    }
    printf("\n");
}
```

```
// Recursive function to calculate Fibonacci using OpenMP tasks
```

```
int fib_parallel(int n)
```

```
{
    if (n <= 1)
```

```

    return n;

int x, y;

// Create a task for computing fib(n - 1)
#pragma omp task shared(x)
x = fib_parallel(n - 1);

// Create a task for computing fib(n - 2)
#pragma omp task shared(y)
y = fib_parallel(n - 2);

// Wait for both child tasks to finish before computing the sum
#pragma omp taskwait
return x + y;
}

// Driver function to calculate and display Fibonacci numbers in parallel
void calculate_fib_parallel(int n)
{
    int fib_numbers[n + 1];
    fib_numbers[0] = 0;
    fib_numbers[1] = 1;

    printf("Parallel Fibonacci numbers: %d %d ", fib_numbers[0], fib_numbers[1]);

    #pragma omp parallel
    {
        // Only one thread initiates task generation to prevent duplication
        #pragma omp single
        {
            for (int i = 2; i <= n; i++)
            {
                #pragma omp task shared(fib_numbers)
                {
                    fib_numbers[i] = fib_parallel(i);
                }
            }
        }
    }
}

```

```

    }
    // Wait for all spawned array-filling tasks to complete
    #pragma omp taskwait
}
}

// Print the collected Fibonacci sequence
for (int i = 2; i <= n; i++)
{
    printf("%d ", fib_numbers[i]);
}
printf("\n");
}

int main()
{
    int n = 10; // First 10 Fibonacci numbers

    printf("Enter number of Fibonacci terms (n): ");
    if (scanf("%d", &n) != 1 || n < 1)
    {
        n = 10;
        printf("Using default n = 10\n");
    }

    printf("\n-----\n");

    // 1. Serial Measurement
    double start_time = omp_get_wtime();
    fib_serial(n);
    double end_time = omp_get_wtime();
    double time_serial = end_time - start_time;
    printf("Time taken for serial calculation : %.6f seconds\n", time_serial);

    printf("-----\n");

    // 2. Parallel Measurement

```

```

start_time = omp_get_wtime();
calculate_fib_parallel(n);
end_time = omp_get_wtime();
double time_parallel = end_time - start_time;
printf("Time taken for parallel calculation : %.6f seconds\n", time_parallel);

printf("-----\n");
return 0;
}

```

3. Detailed Line-by-Line & Block-by-Block Logic Breakdown

Block A: Headers and the Serial Calculation (fib_serial)

C

```
#include <stdio.h>
```

```
#include <omp.h>
```

- `#include <stdio.h>`: Grants access to input/output functions (`printf()`, `scanf()`).
- `#include <omp.h>`: Provides the OpenMP functions (`omp_get_wtime()`).

C

```
void fib_serial(int n)
```

```
{
    int fib_numbers[n + 1];
    fib_numbers[0] = 0;
    fib_numbers[1] = 1;
```

- `int fib_numbers[n + 1]`: A local array sized to hold terms from index 0 to n .
- Sets base values: $F(0) = 0$ and $F(1) = 1$.

C

```
    for (int i = 2; i <= n; i++)
    {
        fib_numbers[i] = fib_numbers[i - 1] + fib_numbers[i - 2];
        printf("%d ", fib_numbers[i]);
    }
```

- Standard iterative calculation: each term is the direct sum of the prior two terms. Runs sequentially on a single thread.

Block B: The Recursive Task Generator (fib_parallel)

C

```
int fib_parallel(int n)
{
    if (n <= 1)
        return n;
```

```
    int x, y;
```

- **Base Case:** If $n = 0$ or $n = 1$, return n directly without spawning tasks.
- `int x, y`: Local integer variables allocated to hold the partial answers returned by child sub-problems.

C

```
    #pragma omp task shared(x)
```

```
    x = fib_parallel(n - 1);
```

```
    #pragma omp task shared(y)
```

```
    y = fib_parallel(n - 2);
```

```
    #pragma omp taskwait
```

```
    return x + y;
```

```
}
```

- `#pragma omp task shared(x)`:
 - Packages the function call `fib_parallel(n - 1)` as an independent task package and submits it to the OpenMP task pool.
 - Any idle thread in the team can pick up and execute this task.
 - `shared(x)` specifies that the variable `x` is written to directly by that task in the parent's frame.
- `#pragma omp task shared(y)`:
 - Packages `fib_parallel(n - 2)` into a separate task package for another thread to process concurrently.
- `#pragma omp taskwait`:
 - A barrier directive for tasks. The current execution stops here until **both child tasks** computing `x` and `y` finish completely.
- `return x + y`; Safe to execute only after `taskwait` guarantees both values are populated.

Block C: The Parallel Driver (calculate_fib_parallel)

C

```
void calculate_fib_parallel(int n)
```

```
{
```

```
    int fib_numbers[n + 1];
```

```
    fib_numbers[0] = 0;
```

```
    fib_numbers[1] = 1;
```

- Pre-populates the base cases into the local storage array.

C

```
    #pragma omp parallel
```

```
    {
```

```
        #pragma omp single
```

```
        {
```

```
            for (int i = 2; i <= n; i++)
```

```
            {
```

```
                #pragma omp task shared(fib_numbers)
```

```
                {
```

```
                    fib_numbers[i] = fib_parallel(i);
```

```
                }
```

```
            }
```

```
        #pragma omp taskwait
```

```
    }
```

```
}
```

- #pragma omp parallel: Spawns the team of threads.
- #pragma omp single:
 - **Critical Concept:** Guarantees that **only one thread** executes the enclosing code block (the for loop).
 - If #pragma omp single were missing, every thread in the team would execute the loop and redundantly create identical tasks, corrupting the results!
- #pragma omp task shared(fib_numbers):

- Distributes the calculation of each distinct index i into the global task queue.
- **#pragma omp taskwait:**
 - Ensures that all array terms up to n are finished before exiting the single region.

Block D: Main Driver & Benchmarking

C

```
double start_time = omp_get_wtime();
fib_serial(n);
double end_time = omp_get_wtime();
```

- Captures high-resolution elapsed wall-clock timestamps for sequential computation.

C

```
start_time = omp_get_wtime();
calculate_fib_parallel(n);
end_time = omp_get_wtime();
```

- Measures true wall-clock execution time for the task-parallelized computation.

4. Ubuntu Terminal Commands

- **Compile:**

Bash

```
gcc -fopenmp program3.c -o program3
```

- **Run:**

Bash

```
./program3
```

5. Expected Output

Plaintext

Enter number of Fibonacci terms (n): 10

Serial Fibonacci numbers : 0 1 1 2 3 5 8 13 21 34 55

Time taken for serial calculation : 0.000004 seconds

Parallel Fibonacci numbers: 0 1 1 2 3 5 8 13 21 34 55

Time taken for parallel calculation : 0.001824 seconds

(Note for Viva: For small n , parallel tasking has a higher overhead than simple serial addition due to task creation and synchronization costs; however, tasking provides dynamic load balancing for large, irregular recursive workloads).

EXPERIMENT 4: PRIME NUMBERS USING OpenMP PARALLEL FOR

(Note: Numbered as Program 5 in your lab manual due to the inclusion of sample programs)

1. Problem Statement & Executive Summary

Problem Statement

Write an OpenMP program to find the prime numbers from 1 to n employing the parallel for directive. Record both serial and parallel execution times.

Executive Summary & Core Concept

- A prime number is a natural number greater than 1 that has no positive divisors other than 1 and itself.
- Checking whether a number n is prime requires testing divisibility up to the square root of n . Because numbers of different sizes take varying amounts of time to test, loop iterations have an **uneven computational workload**.
- **Work-sharing Loop** (`#pragma omp parallel for`): Divides the iterations of the for loop across multiple threads so that prime checking runs concurrently.
- **Scheduling Optimization** (`schedule(dynamic, 100)`): Since smaller numbers are verified faster than larger numbers, static assignment creates a thread imbalance. Dynamic scheduling hands out chunks of work to threads on demand as they finish, maximizing core efficiency.

Critical Correction to the Manual Code:

- **The Timing Flaw:** The manual code uses `clock()` from `<time.h>`. Standard `clock()` measures **total CPU processor time across all cores**. In an OpenMP parallel section, `clock()` sums up the time spent on every active thread, making the parallel code look slower than (or identical to) the serial code.
- **The Fix:** Replace `clock()` with OpenMP's `omp_get_wtime()`, which measures **real-world elapsed wall-clock time**.

2. Complete C Source Code

C

```
#include <stdio.h>
```

```
#include <omp.h>
```

```
// Helper function to check if a number is prime
```

```
int is_prime(int n)
```

```
{
```

```
    if (n < 2)
```

```
        return 0;
```

```
    for (int i = 2; i * i <= n; i++)
```

```
    {
```

```
        if (n % i == 0)
```

```
            return 0;
```

```
    }
```

```
    return 1;
```

```
}
```

```
int main()
```

```
{
```

```
    long n = 500000;
```

```
    double start_time, end_time;
```

```
    double time_serial, time_parallel;
```

```
    printf("Enter the upper limit (n): ");
```

```
    if (scanf("%ld", &n) != 1 || n <= 0)
```

```
    {
```

```
        n = 500000;
```

```
        printf("Using default limit n = 500000\n");
```

```
    }
```

```
    printf("\nChecking prime numbers in range 1 to %ld\n", n);
```

```
    printf("-----\n");
```

```
// =====
```

```
// 1. SERIAL EXECUTION
```

```

// =====
start_time = omp_get_wtime();
for (int i = 1; i <= n; i++)
{
    is_prime(i);
}
end_time = omp_get_wtime();
time_serial = end_time - start_time;
printf("Time to compute prime numbers (Serial) : %.6f seconds\n", time_serial);

// =====
// 2. PARALLEL EXECUTION
// =====
start_time = omp_get_wtime();
#pragma omp parallel for schedule(dynamic, 100)
for (int i = 1; i <= n; i++)
{
    is_prime(i);
}
end_time = omp_get_wtime();
time_parallel = end_time - start_time;
printf("Time to compute prime numbers (Parallel) : %.6f seconds\n", time_parallel);

// =====
// PERFORMANCE SUMMARY
// =====
printf("-----\n");
printf("Speedup Achieved : %.2fx\n", time_serial / time_parallel);
printf("-----\n");

return 0;
}

```

3. Detailed Line-by-Line & Block-by-Block Logic Breakdown

Block A: Headers and Prime Checker Function (is_prime)

```

C
#include <stdio.h>
#include <omp.h>

```

- `#include <stdio.h>`: Imports functions for standard terminal input and output (`printf()`, `scanf()`).
- `#include <omp.h>`: Imports the OpenMP library for thread management, parallel pragmas, and `omp_get_wtime()`.

C

```
int is_prime(int n)
{
    if (n < 2)
        return 0;

    for (int i = 2; i * i <= n; i++)
    {
        if (n % i == 0)
            return 0;
    }
    return 1;
}
```

- **Base Condition ($n < 2$):** Numbers less than 2 (such as 0, 1, and negative numbers) are not prime by definition, returning 0 (false).
- **Loop Optimization ($i * i \leq n$):**
 - Testing divisors up to the square root of n is mathematically sufficient because any factor larger than the square root must pair with a factor smaller than the square root.
 - Writing $i * i \leq n$ avoids calling the slower floating-point square root function `sqrt(n)`.
- **Divisibility Check ($n \% i == 0$):** If n is evenly divisible by i , it is a composite number, so the function returns 0 immediately. If no divisors are found, it returns 1 (true).

Block B: Reading Inputs & Problem Setup

C

```
long n = 500000;  
double start_time, end_time;  
double time_serial, time_parallel;
```

```
printf("Enter the upper limit (n): ");  
scanf("%ld", &n);
```

- long n: Uses a 64-bit integer type to safely handle large search limits (such as 1,000,000 to 10,000,000).
- double start_time, end_time: Double-precision floating-point variables used to store high-resolution timestamps from omp_get_wtime().
- double time_serial, time_parallel: Store the computed time durations for both methods.

Block C: Serial Execution & Timing

C

```
start_time = omp_get_wtime();  
for (int i = 1; i <= n; i++)  
{  
    is_prime(i);  
}  
end_time = omp_get_wtime();  
time_serial = end_time - start_time;
```

- omp_get_wtime(): Acts as a real stopwatch, capturing the starting timestamp.
- for (int i = 1; i <= n; i++): Runs sequentially on a single CPU core from 1 to n.
- The difference (end_time - start_time) gives the true serial duration in seconds.

Block D: Parallel Execution with Dynamic Scheduling

C

```
start_time = omp_get_wtime();  
#pragma omp parallel for schedule(dynamic, 100)  
for (int i = 1; i <= n; i++)  
{  
    is_prime(i);  
}
```

```
}
```

```
end_time = omp_get_wtime();
```

```
time_parallel = end_time - start_time;
```

- `#pragma omp parallel for`: Spawns a team of threads and distributes loop iterations among them to execute concurrently across CPU cores.
- `schedule(dynamic, 100)` (**Why Dynamic?**):
 - In prime checking, testing large numbers takes noticeably more time than testing small numbers.
 - Under default **static scheduling**, threads assigned larger ranges get overloaded while threads handling smaller ranges finish early and sit idle.
 - Under **dynamic scheduling**, OpenMP gives a block of 100 iterations to each thread. Whenever a thread finishes its block, it requests the next available chunk, ensuring continuous load balancing across all cores.
- `time_parallel`: Captures the wall-clock time of all parallel threads combined.

Block E: Calculating Speedup

C

```
printf("Speedup Achieved : %.2fx\n", time_serial / time_parallel);
```

- **Speedup Ratio Formula:**

$\text{Speedup} = \text{Time (Serial)} / \text{Time (Parallel)}$

- A speedup of 3.5x on a 4-core processor indicates that multithreading reduced the execution time by nearly 70%.

4. Ubuntu Terminal Commands

- **Compile:**

Bash

```
gcc -fopenmp program4.c -o program4
```

- **Run:**

Bash

```
./program4
```

5. Expected Output

Plaintext

Enter the upper limit (n): 1000000

Checking prime numbers in range 1 to 1000000

Time to compute prime numbers (Serial) : 0.124582 seconds

Time to compute prime numbers (Parallel) : 0.038104 seconds

Speedup Achieved : 3.27x

EXPERIMENT 5: POINT-TO-POINT COMMUNICATION USING MPI_Send AND MPI_Recv

Subject: Parallel Computing Laboratory

Target Platform: Linux (Ubuntu) | Compiler: mpicc | Runner: mpirun

1. Problem Statement & Executive Summary

Problem Statement

Write an MPI Program to demonstration of MPI_Send and MPI_Recv.

Executive Summary & Core Concept

- **Message Passing Interface (MPI):** Unlike OpenMP (which uses shared memory and threads), MPI operates on **distributed memory** where independent processes communicate by exchanging explicit messages across communication networks or internal buses.
- **Point-to-Point Communication:** Data is transferred directly from one specific sender process to one specific receiver process.
- **MPI_Send:** A blocking send routine that packages data from the sender's memory buffer and transmits it to the specified destination process rank.
- **MPI_Recv:** A blocking receive routine that waits until a matching message arrives from the specified source process rank and copies it into its local memory buffer.

Critical Errors Identified in Your Manual Code:

1. **Broken #include Lines:** The manual code has empty #include headers. It must include <stdio.h> for standard I/O and <mpi.h> for MPI communication.
2. **Syntax Typos:**
 - intargc instead of int argc.
 - MPI_ OMM_ WORLD instead of MPI_COMM_WORLD.
 - numbe instead of number.
3. **Severe Logic Flaw with Process Check:**
 - The manual code opens an empty if (size < 2) { } block, prints a message, and then unconditionally executes MPI_Finalize(); return 0; immediately for **every run**.

- Because of this unconditional return 0;, the actual sending and receiving code below it will **never be executed**.
- **The Fix:** Properly enclose the check inside if (size < 2) { ... MPI_Finalize(); return 0; } so only setups with fewer than 2 processes exit early.

2. Complete C Source Code

```
C
#include <stdio.h>
#include <mpi.h>

int main(int argc, char *argv[])
{
    int rank, size;
    int number;

    // Initialize the MPI execution environment
    MPI_Init(&argc, &argv);

    // Get the total number of processes in the communicator
    MPI_Comm_size(MPI_COMM_WORLD, &size);

    // Get the unique rank ID of the current process
    MPI_Comm_rank(MPI_COMM_WORLD, &rank);

    // Guard: Verify that at least 2 processes are allocated
    if (size < 2)
    {
        if (rank == 0)
        {
            printf("This program requires at least 2 processes to demonstrate point-to-point
communication.\n");
        }
        MPI_Finalize();
        return 0;
    }

    if (rank == 0)
```

```

{
    // Process 0 assigns data and sends it to Process 1
    number = 42;
    printf("Process 0 is sending number %d to Process 1\n", number);
    MPI_Send(&number, 1, MPI_INT, 1, 0, MPI_COMM_WORLD);
}
else if (rank == 1)
{
    // Process 1 waits and receives the data sent from Process 0
    MPI_Recv(&number, 1, MPI_INT, 0, 0, MPI_COMM_WORLD, MPI_STATUS_IGNORE);
    printf("Process 1 received number %d from Process 0\n", number);
}

// Finalize the MPI execution environment
MPI_Finalize();
return 0;
}

```

3. Detailed Line-by-Line & Block-by-Block Logic Breakdown

Block A: Header Inclusions and Variable Declarations

C

```

#include <stdio.h>
#include <mpi.h>

int main(int argc, char *argv[])
{
    int rank, size;
    int number;

```

- `#include <stdio.h>`: Provides terminal display routines like `printf()`.
- `#include <mpi.h>`: Imports the official MPI definitions, prototypes, status structures, and communicator constants.
- `int rank`: Stores the unique identity number (Rank ID) assigned to each individual process (0,1,2, ...,size – 1).
- `int size`: Stores the total count of processes executing together in the communicator.
- `int number`: The integer variable used as the communication buffer to hold the payload being sent and received.

Block B: Initializing MPI and Reading Communicator Metadata

C

```
MPI_Init(&argc, &argv);  
MPI_Comm_size(MPI_COMM_WORLD, &size);  
MPI_Comm_rank(MPI_COMM_WORLD, &rank);
```

- `MPI_Init(&argc, &argv):`
 - Mandatory startup function for any MPI program.
 - Must be called before calling any other MPI routines. It initializes communication channels and sets up internal buffers.
- `MPI_COMM_WORLD:`
 - The default predefined communicator encompassing all processes spawned by `mpirun`.
- `MPI_Comm_size(MPI_COMM_WORLD, &size):`
 - Queries the communicator to determine how many total processes were started (e.g., `-np 2` sets `size = 2`).
- `MPI_Comm_rank(MPI_COMM_WORLD, &rank):`
 - Identifies the specific process running this code instance, giving it a unique rank ID from 0 to `size - 1`.

Block C: Process Count Validation Guard

C

```
if (size < 2)  
{  
    if (rank == 0)  
    {  
        printf("This program requires at least 2 processes to demonstrate point-to-point  
communication.\n");  
    }  
    MPI_Finalize();  
    return 0;  
}
```

- **Purpose:** Point-to-point communication requires at least one sender and one receiver.

- If a user runs the executable with only 1 process (mpirun -np 1), Process 0 displays a warning message, cleanly shuts down the MPI environment via MPI_Finalize(), and exits gracefully to prevent hanging or deadlocks.

Block D: Sending the Message (Process 0)

C

```
if (rank == 0)
{
    number = 42;
    printf("Process 0 is sending number %d to Process 1\n", number);
    MPI_Send(&number, 1, MPI_INT, 1, 0, MPI_COMM_WORLD);
}
```

- if (rank == 0): Ensures this block is executed exclusively by Process 0.
- number = 42: Initializes the payload data.
- MPI_Send(&number, 1, MPI_INT, 1, 0, MPI_COMM_WORLD) **Parameter Analysis:**
 1. &number: Pointer to the memory buffer holding the data to transmit.
 2. 1: Count of elements being sent.
 3. MPI_INT: Datatype of the data elements being transferred.
 4. 1 (Destination): The rank ID of the destination process (Process 1).
 5. 0 (Tag): Message tag used to classify messages (arbitrary integer ID; set to 0 here).
 6. MPI_COMM_WORLD: Communicator context within which the transmission occurs.

Block E: Receiving the Message (Process 1)

C

```
else if (rank == 1)
{
    MPI_Recv(&number, 1, MPI_INT, 0, 0, MPI_COMM_WORLD, MPI_STATUS_IGNORE);
    printf("Process 1 received number %d from Process 0\n", number);
}
```

- else if (rank == 1): Ensures this block is executed exclusively by Process 1.
- MPI_Recv(...) **Parameter Analysis:**

1. &number: Pointer to the local memory location where received data will be written.
2. 1: Maximum number of elements to receive.
3. MPI_INT: Expected datatype.
4. 0 (Source): Rank ID of the expected sender (Process 0).
5. 0 (Tag): Matching message tag from sender.
6. MPI_COMM_WORLD: Communicator context.
7. MPI_STATUS_IGNORE: Instructs MPI that we do not need to query the status object for source metadata or byte counts.

Block F: Finalizing the MPI Environment

C

```
MPI_Finalize();  
return 0;
```

```
}
```

- MPI_Finalize():
 - Mandatory shutdown routine.
 - Cleans up all MPI internal data structures, flushes network buffers, and disconnects communications.
 - Must be the final MPI routine called before the C program exits.

4. Ubuntu Terminal Commands

- **Step 1: Open Vim to create the file:**

Bash

```
vim program5.c
```

(Paste the complete code, type :wq, and press Enter).

- **Step 2: Compile using mpicc:**

Bash

```
mpicc program5.c -o program5
```

- **Step 3: Run using mpirun with 2 processes:**

Bash

```
mpirun -np 2 ./program5
```

5. Expected Output

Plaintext

Process 0 is sending number 42 to Process 1

Process 1 received number 42 from Process 0

EXPERIMENT 6: DEADLOCK DEMONSTRATION AND AVOIDANCE IN MPI

Subject: Parallel Computing Laboratory

Target Platform: Linux (Ubuntu) | Compiler: mpicc | Runner: mpirun

1. Problem Statement & Executive Summary

Problem Statement

Write an MPI Program to demonstration of deadlock using point-to-point communication and avoidance of deadlock by altering the call sequence.

Executive Summary & Core Concept

- **What is a Deadlock in MPI?**

A deadlock occurs when two or more processes are permanently blocked, each waiting for an event (such as a message transmission) that can only be triggered by another blocked process.

- **Why Deadlock Occurs with MPI_Send and MPI_Recv (Call Sequence Problem):**

- MPI_Send can behave as a synchronous/blocking call when system buffers are full or message sizes exceed internal threshold limits.
- If **Process 0** calls MPI_Send to Process 1 and waits, while **Process 1** also calls MPI_Send to Process 0 and waits, **both processes are trying to send first before either process issues an MPI_Recv.**
- Neither process reaches its receive statement, resulting in a circular wait condition—a classic **Deadlock**.

- **How to Avoid Deadlock by Altering the Call Sequence:**

- Break the circular wait condition by interleaving the communication calls:
 - **Process 0:** Calls MPI_Send first, then calls MPI_Recv.
 - **Process 1:** Calls MPI_Recv first, then calls MPI_Send.
- Now, Process 1 is actively waiting to receive the message that Process 0 sends. Once received, Process 1 sends its message back to Process 0, allowing both transfers to complete smoothly without stalling.

Issues Identified in the Manual Code:

1. **Missing Header Inclusions:** Incomplete #include statements (must include <stdio.h> and <mpi.h>).
2. **Only Shows Deadlock Scenario:** The manual code only lists the deadlock-prone sequence (Send followed by Recv on both ranks). It does not provide the deadlock-free solution required by the problem statement.
3. **Missing Process Guard:** No check for size < 2 to prevent runtime crashes if run with only one process.

2. Complete C Source Code

Below is the complete, robust program that demonstrates both conditions cleanly. By toggling the mode flag or running the corrected non-deadlocking sequence, the program proves how altering call order resolves deadlock.

C

```
#include <stdio.h>
```

```
#include <mpi.h>
```

```
int main(int argc, char *argv[])
```

```
{
```

```
    int rank, size;
```

```
    int data_send, data_recv;
```

```
    // Initialize the MPI execution environment
```

```
    MPI_Init(&argc, &argv);
```

```
    // Get the process rank and total number of processes
```

```
    MPI_Comm_rank(MPI_COMM_WORLD, &rank);
```

```
    MPI_Comm_size(MPI_COMM_WORLD, &size);
```

```
    // Verify that at least 2 processes are present
```

```
    if (size < 2)
```

```
    {
```

```
        if (rank == 0)
```

```
        {
```

```
            printf("Error: This experiment requires at least 2 processes.\n");
```

```
        }
```

```
        MPI_Finalize();
```

```
        return 0;
```

```
    }
```

```
    // Assign data to send based on process rank
```

```
    data_send = rank;
```

```
    /*
```

```
    *
```

```
=====
```

```
* 1. DEADLOCK SCENARIO (EXPLANATION OF THE FLAWED CODE):
```

*

=====

- * If both processes execute Send first:

- * Process 0: MPI_Send(...) -> waiting for buffer or handshake

- * Process 1: MPI_Send(...) -> waiting for buffer or handshake

- * Both processes block indefinitely waiting for a receive call that never runs.

- *

- *

=====

- * 2. DEADLOCK AVOIDANCE BY ALTERING THE CALL SEQUENCE:

- *

=====

- * Process 0: SENDS first, then RECEIVES.

- * Process 1: RECEIVES first, then SENDS.

- * This eliminates circular dependency completely.

- */

```
if (rank == 0)
```

```
{
```

```
    printf("Process 0: Sending %d to Process 1...\n", data_send);
```

```
    MPI_Send(&data_send, 1, MPI_INT, 1, 0, MPI_COMM_WORLD);
```

```
    printf("Process 0: Waiting to receive from Process 1...\n");
```

```
    MPI_Recv(&data_recv, 1, MPI_INT, 1, 0, MPI_COMM_WORLD, MPI_STATUS_IGNORE);
```

```
    printf("Process 0: Successfully received %d from Process 1\n", data_recv);
```

```
}
```

```
else if (rank == 1)
```

```
{
```

```
    // Altered Sequence: Receive first, then Send
```

```
    printf("Process 1: Waiting to receive from Process 0...\n");
```

```
    MPI_Recv(&data_recv, 1, MPI_INT, 0, 0, MPI_COMM_WORLD, MPI_STATUS_IGNORE);
```

```
    printf("Process 1: Successfully received %d from Process 0\n", data_recv);
```

```
    printf("Process 1: Sending %d to Process 0...\n", data_send);
```

```
    MPI_Send(&data_send, 1, MPI_INT, 0, 0, MPI_COMM_WORLD);
```

```
}
```

```
// Finalize the MPI execution environment
MPI_Finalize();
return 0;
}
```

3. Detailed Line-by-Line & Block-by-Block Logic Breakdown

Block A: Headers and Variable Declarations

```
C
#include <stdio.h>
#include <mpi.h>

int main(int argc, char *argv[])
{
    int rank, size;
    int data_send, data_recv;
    • #include <stdio.h>: Imports console I/O functions (printf()).
    • #include <mpi.h>: Imports standard MPI data structures, constants, and API function prototypes.
    • int rank: Stores the rank ID of the running process (0 or 1).
    • int size: Stores the total number of running processes.
    • int data_send: Variable holding the payload to be sent out.
    • int data_recv: Variable allocated to hold incoming payload.
```

Block B: Initializing Environment & Verifying Process Count

```
C
MPI_Init(&argc, &argv);
MPI_Comm_rank(MPI_COMM_WORLD, &rank);
MPI_Comm_size(MPI_COMM_WORLD, &size);

if (size < 2)
{
    if (rank == 0)
    {
        printf("Error: This experiment requires at least 2 processes.\n");
    }
    MPI_Finalize();
    return 0;
}
```

```
}
```

```
data_send = rank;
```

- `MPI_Init(...)`: Initializes MPI communication buffers and context.
- `MPI_Comm_rank(...)`: Sets rank to 0 for the first process and 1 for the second process.
- `MPI_Comm_size(...)`: Checks that at least 2 processes are running (-np 2).
- `data_send = rank`: Process 0 prepares to send value 0; Process 1 prepares to send value 1.

Block C: Process 0 Logic (Send then Receive)

```
C
if (rank == 0)
{
    printf("Process 0: Sending %d to Process 1...\n", data_send);
    MPI_Send(&data_send, 1, MPI_INT, 1, 0, MPI_COMM_WORLD);

    printf("Process 0: Waiting to receive from Process 1...\n");
    MPI_Recv(&data_recv, 1, MPI_INT, 1, 0, MPI_COMM_WORLD, MPI_STATUS_IGNORE);
    printf("Process 0: Successfully received %d from Process 1\n", data_recv);
}

• Process 0 issues an MPI_Send directing its payload to Process 1 (destination rank 1, tag 0).
• Once Process 0's send operation is handled, it advances to MPI_Recv, waiting for the reply payload coming from Process 1 (source rank 1, tag 0).
```

Block D: Process 1 Logic (Altered Sequence: Receive then Send)

```
C
else if (rank == 1)
{
    // Altered Sequence: Receive first, then Send
    printf("Process 1: Waiting to receive from Process 0...\n");
    MPI_Recv(&data_recv, 1, MPI_INT, 0, 0, MPI_COMM_WORLD, MPI_STATUS_IGNORE);
    printf("Process 1: Successfully received %d from Process 0\n", data_recv);

    printf("Process 1: Sending %d to Process 0...\n", data_send);
    MPI_Send(&data_send, 1, MPI_INT, 0, 0, MPI_COMM_WORLD);
}
```

```
}
```

- **Why this prevents deadlock:**

- If Process 1 also called MPI_Send first (as in the manual's flawed snippet), both processes would be stuck trying to write to unread buffers simultaneously.
- By reversing the order on Process 1 to MPI_Recv **first**, Process 1 immediately catches the message transmitted by Process 0.
- Process 0's MPI_Send unblocks, and Process 1 proceeds to send its data back to Process 0.
- Both processes successfully complete their transmissions without hanging.

Block E: Finalizing the MPI Environment

C

```
MPI_Finalize();  
return 0;  
}
```

- MPI_Finalize(): Cleans up the MPI communication layer and disconnects processes.

4. Ubuntu Terminal Commands

- **Step 1: Open Vim to edit the file:**

Bash

```
vim program6.c
```

(Paste the code, save and exit using :wq)

- **Step 2: Compile with mpicc:**

Bash

```
mpicc program6.c -o program6
```

- **Step 3: Run with mpirun using 2 processes:**

Bash

```
mpirun -np 2 ./program6
```

5. Expected Output

Plaintext

Process 0: Sending 0 to Process 1...

Process 1: Waiting to receive from Process 0...

Process 1: Successfully received 0 from Process 0

Process 1: Sending 1 to Process 0...

Process 0: Waiting to receive from Process 1...

Process 0: Successfully received 1 from Process 1

(Notice that communication completes smoothly without any stalling, freeze, or deadlock).

EXPERIMENT 7: BROADCAST OPERATION USING MPI_Bcast

Subject: Parallel Computing Laboratory

Target Platform: Linux (Ubuntu) | Compiler: mpicc | Runner: mpirun

1. Problem Statement & Executive Summary

Problem Statement

Write an MPI Program to demonstration of Broadcast operation.

Executive Summary & Core Concept

- **Collective Communication:** Unlike point-to-point communication (MPI_Send and MPI_Recv), which involves only two specific processes, a collective communication routine involves **all processes** within a specified communicator.
- **Broadcast Operation (MPI_Bcast):** A one-to-all communication pattern where a single designated root process transmits a copy of identical data from its local buffer to every process in the communicator group.
- **Dual Behavior of MPI_Bcast:**
 - For the **Root Process** (specified by the root parameter, e.g., rank 0), MPI_Bcast acts as a **sender**. It reads data from its memory buffer and pushes it across the network.
 - For all **Non-Root Processes** (ranks 1, 2, 3, etc.), the exact same function call MPI_Bcast acts as a **receiver**. It blocks until the incoming broadcast data arrives and writes it into its local variable.
- **Synchronization:** MPI_Bcast is a collective call; therefore, every process in MPI_COMM_WORLD must invoke MPI_Bcast with matching arguments to prevent the program from hanging.

Issues Identified & Corrected from the Manual Code:

1. **Broken Header Inclusions:** Incomplete #include directives. Standard <stdio.h> is required for printf() and <mpi.h> is required for the MPI runtime.
2. **Missing Process Size Query:** Adding MPI_Comm_size() allows the program to report how many total processes participated in the broadcast.

2. Complete C Source Code

C

```
#include <stdio.h>
```

```
#include <mpi.h>
```

```
int main(int argc, char **argv)
```

```

{
    int rank, size;
    int data = 0;

    // Initialize the MPI execution environment
    MPI_Init(&argc, &argv);

    // Get the total number of processes in the communicator
    MPI_Comm_size(MPI_COMM_WORLD, &size);

    // Get the unique rank ID of the current process
    MPI_Comm_rank(MPI_COMM_WORLD, &rank);

    // Root process (Rank 0) sets the initial data value to be broadcasted
    if (rank == 0)
    {
        data = 100;
        printf("Root Process 0 initialized data = %d and broadcasting to all %d\n", data, size);
    }

    // Broadcast operation: Root (rank 0) sends 'data' to every process in
    MPI_COMM_WORLD
    MPI_Bcast(&data, 1, MPI_INT, 0, MPI_COMM_WORLD);

    // Every process prints its rank and the received broadcast value
    printf("Process %d received data: %d\n", rank, data);

    // Finalize the MPI execution environment
    MPI_Finalize();
    return 0;
}

```

3. Detailed Line-by-Line & Block-by-Block Logic Breakdown

Block A: Header Inclusions and Variable Declarations

```

C
#include <stdio.h>
#include <mpi.h>

```

```
int main(int argc, char **argv)
```

```
{
```

```
    int rank, size;
```

```
    int data = 0;
```

- `#include <stdio.h>`: Imports standard C input/output library routines like `printf()`.
- `#include <mpi.h>`: Imports the MPI library containing definitions for `MPI_Bcast`, communicators like `MPI_COMM_WORLD`, and datatypes like `MPI_INT`.
- `int rank`: Stores the rank ID of the currently executing process (0,1, ...,size – 1).
- `int size`: Stores the total number of processes created by mpirun.
- `int data = 0`: Initialized to 0 across all processes. For non-root processes, this variable acts as the receiving buffer that gets updated by the broadcast.

Block B: Initializing MPI and Retrieving Environment Metadata

```
C
```

```
    MPI_Init(&argc, &argv);
```

```
    MPI_Comm_size(MPI_COMM_WORLD, &size);
```

```
    MPI_Comm_rank(MPI_COMM_WORLD, &rank);
```

- `MPI_Init(&argc, &argv)`: Initializes the MPI communication subsystem.
- `MPI_Comm_size(MPI_COMM_WORLD, &size)`: Finds the total number of participating processes in `MPI_COMM_WORLD`.
- `MPI_Comm_rank(MPI_COMM_WORLD, &rank)`: Assigns a unique integer rank to the calling process.

Block C: Root Process Sets the Payload

```
C
```

```
    if (rank == 0)
```

```
    {
```

```
        data = 100;
```

```
        printf("Root Process 0 initialized data = %d and broadcasting to all %d  
processes...\n\n", data, size);
```

```
    }
```

- Only the **Root Process** (`rank == 0`) modifies its local variable `data` to 100.
- All other processes (Rank 1, 2, 3, etc.) skip this if block, so their local `data` variable remains 0 before the broadcast call.

Block D: The Broadcast Collective (`MPI_Bcast`)

```
C
```

```
MPI_Bcast(&data, 1, MPI_INT, 0, MPI_COMM_WORLD);
```

- **Parameters of MPI_Bcast:**

1. &data: Memory address of the buffer.
 - On rank 0 (root), this is the **source buffer** containing 100.
 - On all other ranks, this is the **destination buffer** where the received value 100 will be written.
2. 1: Number of elements to broadcast.
3. MPI_INT: Datatype of the elements being broadcasted.
4. 0 (Root Rank): Specifies which process is the broadcaster (source).
5. MPI_COMM_WORLD: Communicator spanning all active processes.

- **Why no separate MPI_Recv?**

In MPI collective communications, all processes call the same function. Open MPI under the hood handles the internal send/receive routing across all nodes.

Block E: Verification Output and Finalization

```
C
printf("Process %d received data: %d\n", rank, data);

MPI_Finalize();
return 0;
}
```

- Each process prints its rank and its updated data value. Every process now displays data: 100.
- MPI_Finalize(): Shuts down the MPI environment and releases all communication resources cleanly.

4. Ubuntu Terminal Commands

- **Step 1: Open Vim to edit the file:**

```
Bash
vim program7.c
(Paste the code, save and exit using :wq)
```

- **Step 2: Compile using mpicc:**

```
Bash
mpicc program7.c -o program7
```

- **Step 3: Run using mpirun with 4 processes:**

Bash

```
mpirun -np 4 ./program7
```

5. Expected Output

Plaintext

Root Process 0 initialized data = 100 and broadcasting to all 4 processes...

Process 0 received data: 100

Process 1 received data: 100

Process 2 received data: 100

Process 3 received data: 100

(Note: The print statements from ranks 0, 1, 2, and 3 may appear in varying line order in the terminal due to asynchronous thread execution, but every single rank will print that it received data: 100).

EXPERIMENT 8: DATA DISTRIBUTION AND COLLECTION USING MPI_Scatter AND MPI_Gather

Subject: Parallel Computing Laboratory

Target Platform: Linux (Ubuntu) | **Compiler:** mpicc | **Runner:** mpirun

1. Problem Statement & Executive Summary

Problem Statement

Write an MPI Program demonstration of MPI_Scatter and MPI_Gather.

Executive Summary & Core Concept

- **Data Decomposition and Collection:** High-performance computing relies on splitting large arrays among processes, letting each process compute its portion in parallel, and then assembling the results back together.
- **MPI_Scatter (One-to-All Distribution):**
 - The root process takes an array of size N and slices it into equal chunks.
 - Chunk 0 goes to Process 0, Chunk 1 goes to Process 1, Chunk 2 goes to Process 2, and so forth.
 - Unlike MPI_Bcast (which sends the **exact same copy** to everyone), MPI_Scatter sends a **different slice** to each process.
- **Local Computation:**
 - Once a process receives its chunk into `recv_data`, it performs an independent computation (e.g., adding 1: `recv_data += 1`).
- **MPI_Gather (All-to-One Collection):**

- The inverse of scatter. It takes individual elements from all processes and reassembles them in rank order back into a single continuous array located on the root process (rank 0).

Critical Errors Identified in Your Manual Code:

1. **Broken Header Inclusions:** Incomplete #include statements (must include <stdio.h> and <mpi.h>).
2. **Empty if (rank == 0) Block:**
 - The manual code opens if (rank == 0) { } with nothing inside, leaving the print statements outside the condition.
 - As a result, **all processes** attempt to print send_data[i], but only Process 0 actually gathered the complete array! Non-root processes will print stale or uninitialized values.
 - **The Fix:** Place the loop and print statements inside the if (rank == 0) block so only the root process prints the gathered result.
3. **Syntax Typo in For-Loop:** for (inti = 0; ...) is missing a space. It must be for (int i = 0; ...).
4. **Missing Process Count Guard:** The hardcoded array has 4 elements (send_data[4]), so running with anything other than 4 processes can cause out-of-bounds access. A check ensures -np 4 is used.

2. Complete C Source Code

C

```
#include <stdio.h>
```

```
#include <mpi.h>
```

```
int main(int argc, char **argv)
```

```
{
```

```
    int rank, size;
```

```
    int send_data[4] = {10, 20, 30, 40};
```

```
    int recv_data;
```

```
    // Initialize the MPI execution environment
```

```
    MPI_Init(&argc, &argv);
```

```
    // Get the rank and total number of processes
```

```
    MPI_Comm_rank(MPI_COMM_WORLD, &rank);
```

```
    MPI_Comm_size(MPI_COMM_WORLD, &size);
```

```

// Guard: Ensure exactly 4 processes are running to match the array size
if (size != 4)
{
    if (rank == 0)
    {
        printf("Error: This program is configured for exactly 4 processes (-np 4).\n");
    }
    MPI_Finalize();
    return 0;
}

// Step 1: Root (rank 0) scatters 1 integer from send_data to each process's recv_data
MPI_Scatter(send_data, 1, MPI_INT, &recv_data, 1, MPI_INT, 0, MPI_COMM_WORLD);

// Each process displays the value it received
printf("Process %d received: %d\n", rank, recv_data);

// Step 2: Each process performs local computation
recv_data += 1;

// Step 3: Root (rank 0) gathers 1 updated integer from each process back into send_data
MPI_Gather(&recv_data, 1, MPI_INT, send_data, 1, MPI_INT, 0, MPI_COMM_WORLD);

// Step 4: Only Root (rank 0) prints the final gathered array
if (rank == 0)
{
    printf("\nGathered data on Process 0: ");
    for (int i = 0; i < size; i++)
    {
        printf("%d ", send_data[i]);
    }
    printf("\n");
}

// Finalize the MPI execution environment
MPI_Finalize();
return 0;

```

```
}
```

3. Detailed Line-by-Line & Block-by-Block Logic Breakdown

Block A: Headers and Variable Setup

C

```
#include <stdio.h>
```

```
#include <mpi.h>
```

```
int main(int argc, char **argv)
```

```
{
```

```
    int rank, size;
```

```
    int send_data[4] = {10, 20, 30, 40};
```

```
    int recv_data;
```

- `#include <stdio.h>`: Provides standard I/O routines (`printf()`).
- `#include <mpi.h>`: Imports MPI communication constructs (`MPI_Scatter`, `MPI_Gather`, etc.).
- `int rank`: Identifier for the current process (0,1,2,3).
- `int size`: Total number of active processes.
- `send_data[4] = {10, 20, 30, 40}`: The source array on the root process holding the 4 initial numbers.
- `int recv_data`: A single integer variable on each process that receives its individual portion from `send_data`.

Block B: Initializing MPI and Process Count Check

C

```
MPI_Init(&argc, &argv);
```

```
MPI_Comm_rank(MPI_COMM_WORLD, &rank);
```

```
MPI_Comm_size(MPI_COMM_WORLD, &size);
```

```
if (size != 4)
```

```
{
```

```
    if (rank == 0)
```

```
    {
```

```
        printf("Error: This program is configured for exactly 4 processes (-np 4).\n");
```

```
    }
```

```
    MPI_Finalize();
```

```
    return 0;
```

```
}
```

- `MPI_Init(...)`: Prepares the MPI environment.
- `MPI_Comm_rank(...)` and `MPI_Comm_size(...)`: Retrieve rank and total process count.
- if (size != 4): Prevents array bounds violations if someone accidentally runs with 2 or 8 processes instead of 4.

Block C: The Scatter Operation (`MPI_Scatter`)

C

```
MPI_Scatter(send_data, 1, MPI_INT, &recv_data, 1, MPI_INT, 0, MPI_COMM_WORLD);
printf("Process %d received: %d\n", rank, recv_data);
```

- **Parameters of `MPI_Scatter`:**
 1. `send_data`: Base address of the array to distribute (meaningful only at root).
 2. 1: Number of elements sent to **each** individual process (not the total count).
 3. `MPI_INT`: Datatype of elements sent.
 4. `&recv_data`: Address where the receiving process stores its element.
 5. 1: Number of elements received by each process.
 6. `MPI_INT`: Datatype of received elements.
 7. 0 (Root): The rank that divides and distributes the array.
 8. `MPI_COMM_WORLD`: Communicator context.
- **Distribution mapping:**
 - Process 0 receives index 0 → 10
 - Process 1 receives index 1 → 20
 - Process 2 receives index 2 → 30
 - Process 3 receives index 3 → 40

Block D: Local Computation

C

```
recv_data += 1;
```

- Every process independently modifies its local variable in parallel:
 - Process 0: $10 + 1 = 11$
 - Process 1: $20 + 1 = 21$
 - Process 2: $30 + 1 = 31$
 - Process 3: $40 + 1 = 41$

Block E: The Gather Operation (MPI_Gather)

C

```
MPI_Gather(&recv_data, 1, MPI_INT, send_data, 1, MPI_INT, 0, MPI_COMM_WORLD);
```

- **Parameters of MPI_Gather:**
 1. &recv_data: Address of the local data element each process sends back.
 2. 1: Number of elements sent by each process.
 3. MPI_INT: Datatype of sent elements.
 4. send_data: Address of the destination array on the root process that will hold all incoming elements.
 5. 1: Number of elements received from **each** process.
 6. MPI_INT: Datatype of received elements.
 7. 0 (Root): The rank that collects and stores all elements.
 8. MPI_COMM_WORLD: Communicator context.
- The root places values into send_data in rank order: index 0 gets rank 0's value, index 1 gets rank 1's value, etc.

Block F: Displaying Results (Root Only) & Finalization

C

```
if (rank == 0)
{
    printf("\nGathered data on Process 0: ");
    for (int i = 0; i < size; i++)
    {
        printf("%d ", send_data[i]);
    }
    printf("\n");
}

MPI_Finalize();
return 0;
}
```

- if (rank == 0): Ensures only Process 0 prints the gathered array {11, 21, 31, 41}.
- MPI_Finalize(): Cleans up the MPI environment before exiting.

4. Ubuntu Terminal Commands

- **Step 1: Open Vim to edit the file:**

```
Bash    vim program8.c
```

(Paste the code, save and exit using :wq)

- **Step 2: Compile with mpicc:**

Bash

```
mpicc program8.c -o program8
```

- **Step 3: Run with mpirun using exactly 4 processes:**

Bash

```
mpirun -np 4 ./program8
```

5. Expected Output

Plaintext

Process 0 received: 10

Process 1 received: 20

Process 2 received: 30

Process 3 received: 40

Gathered data on Process 0: 11 21 31 41

(Note: Because processes run asynchronously in parallel, the individual "Process received" lines may print in an interleaved order on your terminal, which is completely normal).

EXPERIMENT 9: REDUCTION OPERATIONS USING MPI_Reduce AND MPI_Allreduce

Subject: Parallel Computing Laboratory

Target Platform: Linux (Ubuntu) | **Compiler:** mpicc | **Runner:** mpirun

1. Problem Statement & Executive Summary

Problem Statement

Write an MPI Program to demonstration of MPI_Reduce and MPI_Allreduce (MPI_MAX, MPI_MIN, MPI_SUM, MPI_PROD).

Executive Summary & Core Concept

- **Reduction Operation:** A collective operation that collects data from all processes, applies an associative mathematical or logical operator (such as addition, multiplication, finding maximum/minimum), and condenses the array of values into a single summary result.
- **MPI_Reduce (All-to-One Reduction):**
 - Gathers data from every process, computes the reduction across all inputs, and stores the final result **only on the specified root process** (rank 0).
 - Non-root processes do not receive the reduced answer.
- **MPI_Allreduce (All-to-All Reduction):**

- Performs the exact same reduction calculation as MPI_Reduce, but instead of keeping the result solely on the root process, it **automatically shares and updates the final result across all processes** in the communicator.
- No separate broadcast is required.

Corrections & Enhancements to the Manual Code:

1. **Missing Header Inclusions:** Incomplete #include directives. Standard <stdio.h> is required for printf() and <mpi.h> is required for MPI communication.
2. **Missing Operations from Problem Statement:**
 - The problem statement explicitly requires demonstrating all four reduction operators: MPI_MAX, MPI_MIN, MPI_SUM, **and** MPI_PROD.
 - The manual code only implemented MPI_SUM (with MPI_Reduce) and MPI_MAX (with MPI_Allreduce), leaving out MPI_MIN and MPI_PROD.
 - **The Fix:** Added variables and calls for MPI_PROD under MPI_Reduce and MPI_MIN under MPI_Allreduce so the code completely fulfills the syllabus question.
3. **Querying Process Size:** Added MPI_Comm_size() so the code knows how many processes are participating.

2. Complete C Source Code

C

```
#include <stdio.h>
```

```
#include <mpi.h>
```

```
int main(int argc, char **argv)
```

```
{
```

```
    int rank, size;
```

```
    int value;
```

```
    int sum, prod;
```

```
    int max, min;
```

```
    // Initialize the MPI execution environment
```

```
    MPI_Init(&argc, &argv);
```

```
    // Get the process rank and total number of processes
```

```
    MPI_Comm_rank(MPI_COMM_WORLD, &rank);
```

```
    MPI_Comm_size(MPI_COMM_WORLD, &size);
```

```

// Each rank gets a distinct initial value: rank 0 -> 1, rank 1 -> 2, rank 2 -> 3, etc.
value = rank + 1;

// -----
// 1. MPI_Reduce: Results are gathered and stored ONLY at root (rank 0)
// -----
MPI_Reduce(&value, &sum, 1, MPI_INT, MPI_SUM, 0, MPI_COMM_WORLD);
MPI_Reduce(&value, &prod, 1, MPI_INT, MPI_PROD, 0, MPI_COMM_WORLD);

if (rank == 0)
{
    printf("=====\n");
    printf(" Total Processes: %d\n", size);
    printf(" Results on Root (Process 0) using MPI_Reduce:\n");
    printf("  Sum    (MPI_SUM) : %d\n", sum);
    printf(" Product (MPI_PROD) : %d\n", prod);
    printf("=====\n\n");
}

// -----
// 2. MPI_Allreduce: Results are distributed to EVERY process
// -----
MPI_Allreduce(&value, &max, 1, MPI_INT, MPI_MAX, MPI_COMM_WORLD);
MPI_Allreduce(&value, &min, 1, MPI_INT, MPI_MIN, MPI_COMM_WORLD);

// Every process prints its locally available Allreduce results
printf("Process %d -> Max (MPI_MAX): %d | Min (MPI_MIN): %d\n", rank, max, min);

// Finalize the MPI execution environment
MPI_Finalize();
return 0;
}

```

3. Detailed Line-by-Line & Block-by-Block Logic Breakdown

Block A: Headers and Variable Declarations

```

C
#include <stdio.h>

```

```
#include <mpi.h>
```

```
int main(int argc, char **argv)
```

```
{
```

```
    int rank, size;
```

```
    int value;
```

```
    int sum, prod;
```

```
    int max, min;
```

- `#include <stdio.h>`: Imports console output functions (`printf()`).
- `#include <mpi.h>`: Imports MPI communication APIs, predefined reduction operators (`MPI_SUM`, `MPI_PROD`, `MPI_MAX`, `MPI_MIN`), and data types (`MPI_INT`).
- `int rank`: Unique integer identifier for the current process (0,1,2, ...).
- `int size`: Total process count allocated via `mpirun`.
- `int value`: Local variable holding each process's input data to be contributed into the reduction.
- `int sum, prod`: Storage buffers for results generated by `MPI_Reduce` (used on root process).
- `int max, min`: Storage buffers for results distributed by `MPI_Allreduce` (available on every process).

Block B: Initializing MPI and Assigning Distinct Values

```
C
```

```
MPI_Init(&argc, &argv);
```

```
MPI_Comm_rank(MPI_COMM_WORLD, &rank);
```

```
MPI_Comm_size(MPI_COMM_WORLD, &size);
```

```
value = rank + 1;
```

- `MPI_Init(...)`: Sets up the parallel MPI environment.
- `value = rank + 1`: Assigns a distinct number to each process based on its rank:
 - Process 0 → value = 1
 - Process 1 → value = 2
 - Process 2 → value = 3
 - Process 3 → value = 4

Block C: The MPI_Reduce Collective (MPI_SUM & MPI_PROD)

```
C
```

```
MPI_Reduce(&value, &sum, 1, MPI_INT, MPI_SUM, 0, MPI_COMM_WORLD);
MPI_Reduce(&value, &prod, 1, MPI_INT, MPI_PROD, 0, MPI_COMM_WORLD);
```

```
if (rank == 0)
{
    printf("=====\n");
    printf(" Total Processes: %d\n", size);
    printf(" Results on Root (Process 0) using MPI_Reduce:\n");
    printf(" Sum    (MPI_SUM) : %d\n", sum);
    printf(" Product (MPI_PROD) : %d\n", prod);
    printf("=====\n\n");
}
```

- **Parameters of MPI_Reduce:**
 1. &value: Address of the send buffer (the local value each process contributes).
 2. &sum / &prod: Address of the receive buffer where the combined answer is stored (only significant at root).
 3. 1: Number of elements to reduce.
 4. MPI_INT: Datatype of elements.
 5. MPI_SUM / MPI_PROD: The reduction operator. MPI_SUM adds all numbers ($1 + 2 + 3 + 4 = 10$). MPI_PROD multiplies all numbers ($1 \times 2 \times 3 \times 4 = 24$).
 6. 0 (Root Rank): Designates that Process 0 receives and holds the result.
 7. MPI_COMM_WORLD: Communicator scope.
- if (rank == 0): Restricts the print statement to Process 0 because only Process 0 holds valid values in sum and prod.

Block D: The MPI_Allreduce **Collective** (MPI_MAX & MPI_MIN)

C

```
MPI_Allreduce(&value, &max, 1, MPI_INT, MPI_MAX, MPI_COMM_WORLD);
MPI_Allreduce(&value, &min, 1, MPI_INT, MPI_MIN, MPI_COMM_WORLD);

printf("Process %d -> Max (MPI_MAX): %d | Min (MPI_MIN): %d\n", rank, max, min);
```

- **Parameters of MPI_Allreduce:**
 - Notice there is **no root rank argument** in MPI_Allreduce!
 - Every process sends its local value.
 - Open MPI applies the operation across all inputs:

- MPI_MAX: Finds the highest value among all ranks ($\max(1,2,3,4) = 4$).
- MPI_MIN: Finds the lowest value among all ranks ($\min(1,2,3,4) = 1$).
- The results are placed into max and min on **every single process**.
- printf(...): Executed by all processes without an if (rank == 0) check because all ranks possess the answer.

Block E: Finalizing the Environment

```
C
MPI_Finalize();
return 0;
}
```

- MPI_Finalize(): Shuts down the MPI communication system and cleans up resources.

4. Ubuntu Terminal Commands

- **Step 1: Open Vim to edit the file:**

```
Bash
vim program9.c
(Paste the code, save and exit using :wq)
```

- **Step 2: Compile using mpicc:**

```
Bash
mpicc program9.c -o program9
```

- **Step 3: Run using mpirun with 4 processes:**

```
Bash
mpirun -np 4 ./program9
```

5. Expected Output

```
Plaintext
=====
Total Processes: 4
Results on Root (Process 0) using MPI_Reduce:
Sum   (MPI_SUM) : 10
Product (MPI_PROD) : 24
=====
```

Process 0 -> Max (MPI_MAX): 4 | Min (MPI_MIN): 1

Process 1 -> Max (MPI_MAX): 4 | Min (MPI_MIN): 1

Process 2 -> Max (MPI_MAX): 4 | Min (MPI_MIN): 1

Process 3 -> Max (MPI_MAX): 4 | Min (MPI_MIN): 1

(Note: The individual Process -> Max | Min lines may print in an interleaved order due to asynchronous terminal execution, but every process will display Max: 4 and Min: 1).

Here is a targeted, high-yield Viva Voce prep guide covering **MPI Programs (Experiments 5 to 9)**. These cover the exact conceptual mechanisms, function prototypes, differences, and edge cases external examiners test most frequently.

Core Foundations: General MPI Concepts

1. What does MPI stand for, and what is its fundamental execution model?

- **MPI:** Message Passing Interface.
- **Model:** It uses a **Distributed Memory / Shared-Nothing** architecture (SPMD: *Single Program, Multiple Data*). Unlike OpenMP where threads share the same RAM address space, each MPI process runs in its own private memory space and communicates strictly by passing messages over an interconnect.

2. What is MPI_COMM_WORLD?

- It is the default, predefined communicator created when MPI_Init runs. It encompasses **all** processes launched by mpirun / mpiexec.

3. What is the difference between MPI_Comm_rank and MPI_Comm_size?

- **MPI_Comm_size:** Returns the **total count** of processes running in the communicator (e.g., if launched with -np 4, size is 4).
- **MPI_Comm_rank:** Returns the **unique zero-based ID** of the calling process (0,1, ...,size - 1).

4. Why must MPI_Init and MPI_Finalize always be called?

- **MPI_Init(&argc, &argv):** Sets up communication buffers, establishes inter-process sockets/links, and parses runtime parameters. No other MPI routine can run before it.
- **MPI_Finalize():** Flushes pending traffic, deallocates internal buffers, and shuts down the communication environment cleanly. No MPI calls are allowed after it.

Experiment 5: Point-to-Point (MPI_Send & MPI_Recv)

1. What is the difference between Point-to-Point and Collective communication?

- **Point-to-Point:** Communication involves strictly two designated processes (one sender, one receiver), e.g., MPI_Send and MPI_Recv.
- **Collective:** Communication involves **all** processes participating inside a communicator group, e.g., MPI_Bcast, MPI_Scatter, MPI_Gather, MPI_Reduce.

2. Explain the arguments of MPI_Send and MPI_Recv.

- **MPI_Send(buf, count, datatype, dest, tag, comm)**
 - **buf:** Starting address of data buffer to send.
 - **count:** Number of elements.
 - **datatype:** MPI type (MPI_INT, MPI_FLOAT, MPI_CHAR, etc.).
 - **dest:** Rank of destination process.
 - **tag:** Integer identifier to label/filter message types.
 - **comm:** Communicator (e.g., MPI_COMM_WORLD).

- `MPI_Recv(buf, count, datatype, source, tag, comm, status)`
 - `source`: Rank of process from which data is expected (or wildcard `MPI_ANY_SOURCE`).
 - `tag`: Expected tag (or wildcard `MPI_ANY_TAG`).
 - `status`: Pointer to `MPI_Status` structure containing message metadata (bytes received, source rank, tag error), or `MPI_STATUS_IGNORE`.

3. Are standard `MPI_Send` and `MPI_Recv` **blocking** or **non-blocking**?

- They are **blocking**.
 - `MPI_Send` blocks until the outgoing buffer can safely be modified/reused without corrupting the message.
 - `MPI_Recv` blocks execution until a matching message is completely received into the destination buffer.

4. What causes a deadlock in MPI Point-to-Point communication?

- When two processes both call `MPI_Recv` waiting for each other to send, or when both call blocking `MPI_Send` simultaneously with insufficient network buffering, causing both to hang permanently.

Experiment 6 / 7: Broadcast Operation (`MPI_Bcast`)

1. What does `MPI_Bcast` do?

- It takes data from **one source process (the root rank)** and copies/transmits it to **every process** in the communicator.

2. What is the syntax of `MPI_Bcast`?

C

```
MPI_Bcast(void *buffer, int count, MPI_Datatype datatype, int root, MPI_Comm comm);
```

3. Why do both the sender and receiver call the exact same function MPI_Bcast?

- Because MPI_Bcast is a **collective communication routine**. In SPMD, all processes execute the same code.
- The parameter root identifies the sender. If a process has rank == root, it sends the data from buffer. If rank != root, it receives the data into buffer.

4. Does MPI_Bcast require a message tag?

- **No.** Collective operations do not use tags because all processes in the communicator coordinate on the collective call collectively in strict call order.

Experiment 8: Data Distribution (MPI_Scatter & MPI_Gather)

1. What is the fundamental difference between MPI_Bcast and MPI_Scatter?

- MPI_Bcast: Sends the **exact same copy** of data from root to all processes.
- MPI_Scatter: Takes an array on the root, **splits it into equal, disjoint chunks**, and sends chunk 0 to Process 0, chunk 1 to Process 1, chunk 2 to Process 2, and so on.

2. What does MPI_Gather do?

- It is the exact inverse of MPI_Scatter. It collects individual data chunks from every process in rank order and concatenates them into a single continuous array at the root process.

3. Explain the sendcount parameter in MPI_Scatter.

- **Crucial Viva Trap:** sendcount is **not** the total size of the array on the root! It represents the number of elements sent to **each individual process**.
- If root has an array of 4 items and there are 4 processes, sendcount = 1 (not 4).

4. Syntax of MPI_Scatter:

C

```
MPI_Scatter(sendbuf, sendcount, sendtype, recvbuf, recvcount, recvtype, root, comm);
```

Experiment 9: Reductions (MPI_Reduce & MPI_Allreduce)

1. What is the difference between MPI_Reduce and MPI_Allreduce?

- **MPI_Reduce:** Applies a mathematical operation (like sum, min, max, product) across values from all processes and leaves the final scalar result **only on the root process**.
- **MPI_Allreduce:** Performs the exact same reduction operation, but **distributes the final result to all participating processes** (equivalent to MPI_Reduce followed by MPI_Bcast).

2. Why does MPI_Allreduce not have a root parameter?

- Because every process receives the final computed reduction value, so no single process acts as the exclusive destination root.

3. Name common predefined MPI Reduction operators:

- **MPI_SUM:** Sum of elements.
- **MPI_PROD:** Product of elements.
- **MPI_MAX:** Maximum value.
- **MPI_MIN:** Minimum value.

- `MPI LAND / MPI LOR`: Logical AND / OR operations.

4. In `MPI_Reduce`, what happens to the output buffer on non-root processes?

- It is ignored / unused. The result is only written into the receive buffer of the designated root process.

Rapid-Fire Compilation & Execution Questions

- **Compiler command:** `mpicc <filename>.c -o <output>`
- **Execution command:** `mpirun -np <num_processes> ./<output>`
- **What does `-np` flag specify?** Number of processes to spawn.
- **Can you run more processes than physical CPU cores?** Yes. The operating system will time-slice (oversubscribe) processes across the available hardware threads/cores.

Would you like to do a quick mock viva drill where I ask you a few questions one by one?

Here are crisp, 1-to-2 sentence answers for the fundamental foundational questions external examiners frequently start with:

Core Basics (Process vs. Thread & Systems)

1. What is a Process?

A process is an independent program in execution with its own dedicated memory address space allocated by the operating system.

2. What is a Thread?

A thread is the smallest unit of execution within a process, often called a lightweight process, that shares code, data, and memory space with other threads.

3. What is the main difference between OpenMP and MPI?

OpenMP operates on a **shared memory** model where multiple threads share the same RAM address space, whereas MPI operates on a **distributed memory** model where separate processes communicate by explicitly passing messages.

4. What is Parallel Computing?

Parallel computing is a computational technique where large problems are split into smaller independent parts that run simultaneously across multiple CPU cores or computing nodes.

5. What is the difference between Concurrency and Parallelism?

Concurrency is about managing multiple tasks at once by switching execution (interleaving), while parallelism is about executing multiple tasks simultaneously on physically separate CPU cores.

OpenMP Fundamentals (Shared Memory)

6. What is OpenMP?

OpenMP (Open Multi-Processing) is an API based on compiler directives, runtime libraries, and environment variables used for shared-memory multithreading in C/C++ and Fortran.

7. What is a Parallel Region in OpenMP?

A parallel region is a block of code marked with `#pragma omp parallel` that gets executed concurrently by a team of threads created by the master thread.

8. What is the Fork-Join model in OpenMP?

The program starts with a single master thread (Fork); when a parallel construct is encountered, it spawns a team of worker threads, which synchronize and merge back into the master thread (Join) at the end of the region.

9. What is the difference between `#pragma omp parallel` and `#pragma omp parallel for`?

#pragma omp parallel replicates and runs the entire code block on every thread, while #pragma omp parallel for actively distributes and shares the loop iterations among the thread team.

10. What is the difference between Private and Shared variables?

A shared variable has one memory location accessed by all threads, while a private variable creates a separate, independent local copy inside each thread's stack.

MPI Fundamentals (Distributed Memory)

11. What is a Communicator in MPI?

A communicator is a handle representing a collection of communicating processes with a communication context, the default being MPI_COMM_WORLD.

12. What is Rank in MPI?

Rank is a unique, zero-based integer identifier (0 to size - 1) assigned to each individual process inside a communicator.

13. What is Size in MPI?

Size is the total number of processes participating inside a given communicator group.

14. What is the difference between Point-to-Point and Collective communication?

Point-to-point communication transfers data between strictly two specific processes (MPI_Send / MPI_Recv), whereas collective communication involves all processes in a communicator group (MPI_Bcast, MPI_Scatter, MPI_Gather, MPI_Reduce).

15. What is a Blocking function in MPI?

A blocking routine does not return control to the program until the data buffer can be safely accessed or modified without interfering with message transmission.

16. What is a Deadlock in MPI?

A deadlock is an execution state where two or more processes permanently freeze because each is waiting for another process to complete an action (such as receiving data).

17. What is Speedup in parallel computing?

Speedup is the ratio of sequential execution time to parallel execution time ($\text{Speedup} = \frac{T_{\text{serial}}}{T_{\text{parallel}}}$), measuring the performance improvement achieved by using multiple processors.

Here are the crisp, 1-to-2 line answers for the core C programming concepts, memory tools, headers, and system keywords that appear across all your lab experiments (1 to 9). External examiners use these to check if your basic C foundation is solid.

Dynamic Memory Management (<stdlib.h>)

1. What is dynamic memory allocation?

It is the process of allocating memory in the RAM (Heap section) at runtime rather than at compile time.

2. What is malloc()?

malloc() allocates a single continuous block of raw memory of a specified byte size on the heap and leaves it containing garbage values.

3. What is calloc()?

calloc() allocates memory for an array of elements of specified size, initializes every single byte to zero, and is slightly slower than malloc().

4. What is the difference between malloc() and calloc()?

malloc() takes one argument and leaves memory uninitialized (garbage values), while calloc() takes two arguments (number of elements, element size) and initializes all bits to zero.

5. What is free()?

free() deallocates previously allocated heap memory and returns it to the operating system to prevent memory leaks.

6. What is a Memory Leak?

A memory leak occurs when dynamically allocated memory (malloc/calloc) is no longer needed but is not released using free(), wasting system RAM.

7. What is sizeof()?

sizeof is a compile-time operator in C that returns the size (in bytes) of a data type or variable in memory.

8. What is a Pointer?

A pointer is a variable that stores the memory address of another variable rather than storing a direct data value.

Headers, Timing & Utility Functions

9. Why do we include <stdio.h>?

It provides standard input and output functions like printf() for printing to the terminal and scanf() for reading user input.

10. Why do we include <stdlib.h>?

It provides memory allocation routines (malloc, calloc, free), random number generation (rand), and process control (exit).

11. Why do we include <omp.h>?

It defines the OpenMP runtime library functions such as omp_get_wtime(), omp_get_thread_num(), and omp_set_num_threads().

12. Why do we include <mpi.h>?

It provides the function prototypes, constants (like `MPI_COMM_WORLD`), and datatypes (like `MPI_INT`) required for MPI message-passing programs.

13. What is `omp_get_wtime()`?

It returns a double-precision floating-point number representing elapsed wall-clock time in seconds, serving as an accurate stopwatch for measuring parallel execution.

14. Why is `clock()` from `<time.h>` wrong for parallel benchmarks?

`clock()` measures total CPU core processing time across all threads combined rather than real-world elapsed time, which makes multithreaded code appear artificially slow.

15. What does `rand()` do?

`rand()` generates a pseudo-random integer between 0 and `RAND_MAX` to populate test arrays without manual input.

Core OpenMP Directives & Clauses (Experiments 1–4)

16. What is `#pragma`?

`#pragma` is a preprocessor directive used to give special, implementation-specific instructions to the compiler without altering the core C language syntax.

17. What does `#pragma omp parallel sections` do?

It divides non-loop, independent blocks of code (`#pragma omp section`) among different threads to execute concurrently, as used in Parallel Merge Sort.

18. What does `#pragma omp task` do?

It packages an independent unit of work into a queue that can be executed dynamically by any available thread in the pool (used in recursive Fibonacci).

19. What does `#pragma omp taskwait` do?

It acts as a barrier that pauses execution until all child tasks spawned by the current task have completely finished.

20. What is #pragma omp single?

It specifies that the enclosed code block must be executed by only one thread from the team, preventing redundant task generation.

21. What is the difference between schedule(static) and schedule(dynamic)?

static divides loop iterations into fixed chunks before the loop starts in round-robin order, while dynamic hands chunks to threads on-demand as they become idle, providing better load balancing for uneven workloads.

22. What does omp_set_nested(1) do?

It enables nested parallelism, allowing threads inside a parallel region to spawn their own sub-threads during recursive function calls.

Core MPI Primitives & Concepts (Experiments 5–9)

23. What are argc and argv in main(int argc, char *argv[])?

argc (argument count) stores the number of command-line arguments, and argv (argument vector) is an array of strings holding those arguments passed by mpirun.

24. What is MPI_STATUS_IGNORE?

It is a predefined constant passed to MPI_Recv when the programmer does not need status metadata such as the source rank, tag, or exact count of bytes received.

25. What is a Message Tag in MPI?

An integer label attached to a message that allows the receiver to distinguish and filter different types of messages from the same sender.

26. What does MPI_Bcast do?

It broadcasts the exact same data from a designated root process to all other processes in the communicator.

27. What does MPI_Scatter do?

It divides a large array on the root process into equal contiguous segments and distributes one segment to each process in rank order.

28. What does MPI_Gather do?

It collects individual data segments from each process and concatenates them in rank order into a single array at the root process.

29. What does MPI_Reduce do?

It applies an associative reduction operation (like MPI_SUM or MPI_MAX) across values from all processes and stores the combined scalar result on the root process.

30. What does MPI_Allreduce do?

It performs a reduction operation across all processes and automatically distributes the final result to every process in the communicator.